

UNIVERSIDAD DEL VALLE DE GUATEMALA

CC3067 Redes



Chatbot vuelos

Proyecto 1

Silvia Alejandra Illescas Fernandez, 22376

Guatemala, 22 de septiembre del 2025

Pruebas

EJEMPLOS LLM (chat general + contexto)

```
Tú: ¿Quién fue Alan Turing?
Chatbot: Alan Turing fue un matemático, lógico y criptógrafo británico conocido por ser uno de los padres de la informática y pionero en el campo de la inteligencia artificial. Durante la Segunda Guerra Mundial, Turing trabajó en descifrar códigos secretos alemanes, contribuyendo de manera significativa a la victoria de los Aliados. También es famoso por crear la "Máquina de Turing", un concepto fundamental en la teoría de la computación. Lamentablemente, Turing fue perseguido por su homosexualidad en la sociedad conservadora de su época y tragado con la castración química, lo que resultó en su trágica muerte en 1954. En la actualidad, es reconocido como un genio y su legado sigue vivo en la informática y la inteligencia artificial.
Tú: ¿En qué fecha nació?
Chatbot: Alan Turing nació el 23 de junio de 1912 en Maida Vale, Londres, Reino Unido.
```

VUELOS (formato libre y formato comando)

```
Tú: buscar vuelo Guatemala Panama 25-09-2025

Vuelos encontrados:
- Copa Airlines CM123 | Salida: 2025-09-25T08:30:00 | Llegada: 2025-09-25T10:00:00 | Estado: scheduled
  Clima en destino: 25.9°C, light rain
  Actividades sugeridas:
    - Canal de Panamá (Rating: 4.8)
    - Casco Viejo (Rating: 4.6)
    - Biomuseo (Rating: 4.5)

- Avianca AV456 | Salida: 2025-09-25T12:00:00 | Llegada: 2025-09-25T13:30:00 | Estado: scheduled
  Clima en destino: 25.9°C, light rain
  Actividades sugeridas:
    - Canal de Panamá (Rating: 4.8)
    - Casco Viejo (Rating: 4.6)
    - Biomuseo (Rating: 4.5)
```

```
Tú: buscar vuelo Guatemala Panama 25-09-2025

Vuelos encontrados:
- Copa Airlines CM123 | Salida: 2025-09-25T08:30:00 | Llegada: 2025-09-25T10:00:00 | Estado: scheduled
  Clima en destino: 25.9°C, light rain
  Actividades sugeridas:
    - Canal de Panamá (Rating: 4.8)
    - Casco Viejo (Rating: 4.6)
    - Biomuseo (Rating: 4.5)

- Avianca AV456 | Salida: 2025-09-25T12:00:00 | Llegada: 2025-09-25T13:30:00 | Estado: scheduled
  Clima en destino: 25.9°C, light rain
  Actividades sugeridas:
    - Canal de Panamá (Rating: 4.8)
    - Casco Viejo (Rating: 4.6)
    - Biomuseo (Rating: 4.5)
```

FILESYSTEM (crear, leer, listar)

```
Tú: listar archivos
Archivos en MCP Filesystem: []
Tú: crear archivo README.md Bienvenido a mi proyecto MCP
Archivo 'README.md' creado en MCP Filesystem.
Tú: leer archivo README.md
Contenido de README.md:
Bienvenido a mi proyecto MCP
Tú: crear archivo "mi nota.txt" Hola con espacios en el nombre
Archivo 'mi nota.txt' creado en MCP Filesystem.
Tú: leer archivo "mi nota.txt"
Contenido de mi nota.txt:
Hola con espacios en el nombre
Tú: crear archivo config.json {"api":"ok","v":1}
Archivo 'config.json' creado en MCP Filesystem.
Tú: leer archivo config.json
Contenido de config.json:
{"api":"ok","v":1}
Tú: listar archivos
Archivos en MCP Filesystem: ['README.md', 'config.json', 'mi nota.txt']
```

GIT (local real)

```
Archivos en MCP Filesystem: ['README.md', 'config.json', 'mi nota.txt']
Tú: git commit README.md "Primer commit del README"
[main 00c1137] "Primer commit del README"
 1 file changed, 1 insertion(+), 1 deletion(-)
Archivo 'README.md' agregado y commit realizado con mensaje: '"Primer commit del README"'
```

CHESS (Stockfish)

```
cuarto hoy:
Tú: analizar ajedrez rnbqkb1r/pppp1ppp/4pn2/8/2B5/5N2/PPPP1PPP/RNBQK2R w KQkq - 0 4
bestmove c4e2 ponder c7c5

Últimas líneas de análisis:
info depth 8 seldepth 10 multipv 1 score cp -162 nodes 1365 nps 195000 hashfull 0 tbhits 0 time 7 pv d2d4 c7c
5 e1g1 c5d4 f3d4 d7d5 c4e2 c8d7
info depth 9 seldepth 11 multipv 1 score cp -144 nodes 1958 nps 217555 hashfull 0 tbhits 0 time 9 pv d2d4 c7c
5 e1g1 c5d4 f3d4
info depth 10 seldepth 15 multipv 1 score cp -168 nodes 4986 nps 207750 hashfull 1 tbhits 0 time 24 pv d2d4 d
7d5 c4d3 c7c5 d4c5 f8c5 e1g1 e8g8
info depth 11 seldepth 15 multipv 1 score cp -137 nodes 7723 nps 208729 hashfull 2 tbhits 0 time 37 pv d2d4 d
7d5 c4e2 c7c5 e1g1 c5d4 f3d4 e6e5 f1e1 f8d6 d4b5 d6e7 c2c4 e8g8
info depth 12 seldepth 25 multipv 1 score cp -142 nodes 20093 nps 216053 hashfull 6 tbhits 0 time 93 pv c4e2
c7c5 c2c4 d7d5 c4d5 f6d5 d2d4 c5d4 f3d4 f8e7 e1g1 e8g8 e2f3
Tú: analizar ajedrez e4 e5 Nf3 Nc6 Bb5
bestmove a7a6 ponder b5a4

Últimas líneas de análisis:
info depth 8 seldepth 9 multipv 1 score cp -29 nodes 5631 nps 351937 hashfull 0 tbhits 0 time 16 pv a7a6 b5a4
f8c5 e1g1 b7b5 a4b3
info depth 9 seldepth 19 multipv 1 score cp -51 nodes 18256 nps 338074 hashfull 7 tbhits 0 time 54 pv a7a6 b5
a4 f8c5 e1g1 b7b5 a4b3 c5b6 d2d4 c6d4 f3d4 b6d4 a2a4
info depth 10 seldepth 14 multipv 1 score cp -51 nodes 19947 nps 343913 hashfull 7 tbhits 0 time 58 pv a7a6 b
5a4 f8c5 e1g1 g8e7 c2c3 c5a7 d2d4 e7g6 c1e3
info depth 11 seldepth 15 multipv 1 score cp -44 nodes 24339 nps 338041 hashfull 8 tbhits 0 time 72 pv g7g6 d
2d4 c6d4 f3d4
info depth 12 seldepth 17 multipv 1 score cp -38 nodes 44457 nps 322152 hashfull 13 tbhits 0 time 138 pv a7a6
b5a4 g8f6 e1g1 b7b5 a4b3 f6e4 f1e1
Tú: □
```

MCP REMOTO (hora)

```
Tú: hora actual
Consultando al MCP remoto de hora...
{'date': '2025-09-22', 'time': '22:29:40', 'timezone': 'America/Guatemala'}
Tú: 
```

LOG E INTERACCIÓN

```
=== LOG DE INTERACCIONES ===
[
  {
    "timestamp": "2025-09-22 22:22:38.636038",
    "endpoint": "/get_flights",
    "request": {
      "origin": "GUA",
      "destination": "PTY",
      "departure_date": "2025-09-25"
    },
    "response": {
      "detail": "Not Found"
    }
  },
  {
    "timestamp": "2025-09-22 22:27:21.242689",
    "endpoint": "MCP Git",
    "request": {
      "file": "README.md",
      "message": "\"Primer commit del README\""
    },
    "response": "Commit realizado"
  }
]
=====
```

```
Tú: salir
¡Hasta luego! 🙋
```

Análisis de la comunicación entre el servidor remoto y el cliente

(ip.addr == 104.16.231.132 ip.addr == 104.16.230.132 ipv6.addr == 2606:4700:6810:e784 ipv6.addr == 2606:4700:6810:e684) && tcp.port == 443									
No.	Time	Source	Destination	Protocol	TCP Segment Len	Info			
312	14:48:51.084468	2800:98:1116:b7e:51...	2606:4700::6810:e784	TCP	0	59788 → 443 [SYN] Seq=0 Win=65535 Len=0 MSS=1440 WS=256 SACK_PERM			
313	14:48:51.155249	2606:4700::6810:e784	2800:98:1116:b7e:51...	TCP	0	443 → 59788 [SYN, ACK] Seq=0 Ack=1 Win=65535 Len=0 MSS=1360 SACK_PERM WS=8192			
314	14:48:51.155383	2800:98:1116:b7e:51...	2606:4700::6810:e784	TCP	0	59788 → 443 [ACK] Seq=1 Ack=1 Win=65280 Len=0			
317	14:48:51.218389	2800:98:1116:b7e:51...	2606:4700::6810:e784	TLShv1.3	517	Client Hello (SNI=roland-custom-big-rehab.trycloudflare.com)			
319	14:48:51.286413	2606:4700::6810:e784	2800:98:1116:b7e:51...	TCP	0	443 → 59788 [ACK] Seq=1 Ack=518 Win=131072 Len=0			
320	14:48:51.290823	2606:4700::6810:e784	2800:98:1116:b7e:51...	TLShv1.3	1360	Server Hello, Change Cipher Spec			
321	14:48:51.291362	2606:4700::6810:e784	2800:98:1116:b7e:51...	TCP	1360	443 → 59788 [ACK] Seq=1361 Ack=518 Win=131072 Len=1360 [TCP PDU reassembled in 323]			
322	14:48:51.291477	2800:98:1116:b7e:51...	2606:4700::6810:e784	TCP	0	59788 → 443 [ACK] Seq=518 Ack=2721 Win=65280 Len=0			
323	14:48:51.292229	2606:4700::6810:e784	2800:98:1116:b7e:51...	TLShv1.3	138	Application Data			
324	14:48:51.299860	2800:98:1116:b7e:51...	2606:4700::6810:e784	TLShv1.3	80	Change Cipher Spec, Application Data			
325	14:48:51.300917	2800:98:1116:b7e:51...	2606:4700::6810:e784	TLShv1.3	202	Application Data			
326	14:48:51.369779	2606:4700::6810:e784	2800:98:1116:b7e:51...	TCP	0	443 → 59788 [ACK] Seq=2859 Ack=800 Win=131072 Len=0			
335	14:48:51.802714	2606:4700::6810:e784	2800:98:1116:b7e:51...	TLShv1.3	855	Application Data, Application Data			
336	14:48:51.803209	2606:4700::6810:e784	2800:98:1116:b7e:51...	TLShv1.3	37	Application Data			
337	14:48:51.803307	2800:98:1116:b7e:51...	2606:4700::6810:e784	TCP	0	59788 → 443 [ACK] Seq=800 Ack=3751 Win=64256 Len=0			
338	14:48:51.803722	2606:4700::6810:e784	2800:98:1116:b7e:51...	TLShv1.3	27	Application Data			
339	14:48:51.805803	2800:98:1116:b7e:51...	2606:4700::6810:e784	TCP	0	59788 → 443 [FIN, ACK] Seq=800 Ack=3778 Win=64256 Len=0			
340	14:48:51.878477	2606:4700::6810:e784	2800:98:1116:b7e:51...	TCP	0	443 → 59788 [FIN, ACK] Seq=3778 Ack=801 Win=131072 Len=0			
341	14:48:51.878557	2800:98:1116:b7e:51...	2606:4700::6810:e784	TCP	0	59788 → 443 [ACK] Seq=801 Ack=3779 Win=64256 Len=0			

Se realizó una captura de tráfico de red en la interfaz Wi-Fi utilizando Wireshark, con el objetivo de analizar las interacciones entre el cliente y el servidor remoto al consultar la hora actual mediante un servicio expuesto a través de un túnel de Cloudflare (roland-custom-big-rehab.trycloudflare.com).

Durante la captura se identificaron las fases propias del intercambio bajo el protocolo TCP/TLS y las operaciones de JSON-RPC que ocurren sobre dicho canal seguro.

Fase de sincronización

En esta etapa se establece la comunicación entre cliente y servidor.

- Se observan los paquetes SYN, SYN/ACK y ACK, que corresponden al handshake TCP.
- Posteriormente se ejecuta el handshake TLS, donde el cliente envía un *Client Hello* (incluyendo el SNI con el dominio del túnel) y el servidor responde con un *Server Hello* y parámetros de cifrado.

Esta fase corresponde a los mensajes de sincronización, ya que permiten iniciar la sesión segura para el intercambio de datos.

Fase de solicitud

Una vez asegurada la comunicación, el cliente envía el primer paquete de Application Data. Aunque el contenido viaja cifrado, se sabe que corresponde a la solicitud JSON-RPC, en este caso el método GET /get_time, que permite obtener la hora actual desde el servidor. Estos paquetes se clasifican como mensajes de solicitud o petición.

Fase de respuesta

El servidor responde inmediatamente con varios paquetes de Application Data, también cifrados bajo TLS. En este flujo se encuentra la respuesta JSON que contiene la hora actual, enviada como objeto JSON-RPC. Estos paquetes constituyen los mensajes de respuesta.

Cierre de la conexión

Finalmente, se observa el intercambio de paquetes FIN y ACK, que indican la terminación de la sesión TCP entre cliente y servidor.

Clasificación de los mensajes observados

Fase	Paquetes observados	Dirección	Descripción general	Tipo JSON-RPC
Sincronización	Handshake TCP/TLS	Cliente ↔ Servidor	Establecimiento de conexión segura	Sincronización
Solicitud	Application Data	Cliente → Servidor	Petición GET /get_time enviada por el cliente	Solicitud/Petición
Respuesta	Application Data	Servidor → Cliente	Respuesta JSON con la hora actual	Respuesta
Cierre conexión	FIN, ACK	Cliente ↔ Servidor	Terminación ordenada de la sesión TCP	—

El análisis de la traza en Wireshark evidencia el ciclo completo de comunicación entre cliente y servidor: sincronización → solicitud → respuesta → cierre. A pesar de que los datos JSON viajan cifrados bajo TLS y no son legibles directamente, la dirección y secuencia de los paquetes permiten clasificar cada fase de manera clara y asociarla con los mensajes JSON-RPC correspondientes.

Especificación de los servidores MCP desarrollados

9.1 MCP Local – “Vuelos” (FastAPI en 127.0.0.1:8000)

Tecnologías: Python 3.11, FastAPI, requests.

Propósito: dada una ruta (IATA→IATA + fecha), retorna vuelos con clima y actividades sugeridas.

Endpoints

- **POST /get_flights**

Body (JSON):

```
{  
  "origin": "GUA",  
  "destination": "MEX",  
  "departure_date": "2025-10-01"  
}
```

- origin: IATA (p.ej., GUA)
- destination: IATA (p.ej., MEX)
- departure_date: YYYY-MM-DD

Respuesta 200 (JSON, ejemplo):

```
{  
  "flights": [  
    {  
      "airline": "Example Air",  
      "flight_number": "EA123",  
      "departure_time": "2025-10-01T07:30:00-06:00",  
      "arrival_time": "2025-10-01T10:55:00-05:00",  
      "status": "scheduled",  
      "weather": { "temperature": 22, "condition": "clear sky" },  
      "activities": [  

```

```

    { "name": "Museo X", "rating": 4.6 },
    { "name": "Parque Y", "rating": 4.4 }
  ]
}
]
}

```

Errores:

- 502 (problema con API externa de vuelos):
{"error": "aviationstack_upstream", "detail": "<mensaje de red/timeout/cuota>"}
- 500 (fallo interno no controlado):
{"error": "server", "detail": "<excepción>"}
- GET /docs – documentación Swagger.
- GET /health *(si lo añadiste)*: {"ok": true}

Notas

- Encapsula la llamada a la API externa (AviationStack) con try/except y devuelve error controlado (no derribar el server).
- Content-Type: application/json.

9.2 MCP Remoto – “Hora” (FastAPI en 127.0.0.1:8787 + Cloudflare Tunnel)

Tecnologías: Python 3.11, FastAPI, pytz/zoneinfo, Cloudflare Tunnel (QUIC).

Propósito: entregar hora oficial según zona horaria (por defecto America/Guatemala).

Endpoints

GET /get_time

Query: tz (opcional, IANA TZ, p.ej., America/Guatemala).

Respuesta 200 (JSON):

```

{
  "timezone": "America/Guatemala",
  "datetime_iso": "2025-09-22T21:31:14-06:00",
  "date": "2025-09-22",
  "time": "21:31:14",
  "utc_offset": "-06:00"
}

```

}

GET /: ping/boilerplate (mensaje de servicio activo).

GET /docs: documentación Swagger

GET /get_time (Cloudflare Worker) → puede responder:

```
{ "time": "2025-09-23T03:28:58.707Z" }
```

El cliente convierte a America/Guatemala si hace falta.

Publicación por túnel

- Comando típico local: `uvicorn remote_mcp:app --host 127.0.0.1 --port 8787 --reload`
- Túnel: `cloudflared tunnel --url http://127.0.0.1:8787`
URL pública efímera:
`https://<subdominio>.trycloudflare.com/get_time?tz=America/Guatemala`
-

9.3 (Integración externa, no desarrollada) – “Log Analyzer” de compañero (FastAPI en 127.0.0.1:8001)

- **GET /status** → `{"status":"OK"}`
- **POST /analyze_log_file** (multipart/form-data, campo file binario).
El bot también soporta enviar “texto como archivo” en UTF-8.

10) Explicación por capas (enlace, red, transporte y aplicación)

10.1 Tráfico local (loopback) — POST 127.0.0.1:8000/get_flights

En capa de enlace (L2) no hay tráfico físico porque el loopback es una interfaz virtual; por ello no aparecen tramas 802.11/Ethernet ni ARP. En red (L3) se utiliza la dirección IPv4 de loopback 127.0.0.1, sin NAT ni enrutamiento externo. En transporte (L4) la conexión TCP hacia el puerto 8000 realiza el *three-way handshake* (SYN → SYN/ACK → ACK) y, al terminar, el lado cliente queda un tiempo en TIME_WAIT; esto es normal y se observa con netstat. En aplicación (L7) el cliente (curl/requests) habla HTTP/1.1 con uvicorn; las peticiones de vuelos se envían como application/json y la API responde con JSON y estado 200 si todo va bien, o con errores controlados 502/500 cuando falla un proveedor externo.

10.2 Tráfico remoto por Cloudflare Tunnel — GET https://<subdominio>.trycloudflare.com/get_time?...

Aquí coexisten dos trayectos. Primero, el túnel saliente desde la máquina local hacia el *edge* de Cloudflare: en L2 viaja por la red local (Wi-Fi/Ethernet) hasta el router; en L3 sale con la IP pública del ISP hacia direcciones de Cloudflare; en L4 usa QUIC sobre UDP 443; y en L7 cloudflared mantiene una sesión cifrada (TLS 1.3/HTTP/3) persistente con el *edge*. Segundo, la solicitud del cliente a la URL pública: el cliente resuelve el subdominio vía DNS y llega al *edge* por HTTPS (HTTP/2 o HTTP/3).

Cloudflare inyecta esa solicitud dentro del túnel activo, y cloudflared la reenvía localmente por HTTP (habitualmente HTTP/1.1 sobre TCP) a 127.0.0.1:8787, donde responde FastAPI; el *edge* entrega luego la respuesta al cliente. En la práctica, esto da una latencia estable mientras el túnel esté saludable. Si el quick tunnel expira (propio de su naturaleza efímera), la URL deja de resolver/servir y hay que relanzarlo o migrar a un named tunnel.

10.3 Tráfico local hacia el MCP externo del compañero — POST 127.0.0.1:8001/analyze_log_file

De nuevo, en L2 no hay tramas físicas por tratarse de loopback; en L3 se usa 127.0.0.1; y en L4 la conexión TCP va al puerto 8001 (servidor uunicorn). En L7 el endpoint exige multipart/form-data con un campo file. Aquí es relevante la codificación: crear archivos con > en PowerShell suele producir UTF-16, mientras que el servidor espera UTF-8; de ahí el error interno con el mensaje del códec UTF-8. La corrección es generar el archivo explícitamente en UTF-8 (por ejemplo, Set-Content -Encoding utf8) o enviar el contenido como archivo en UTF-8 desde el cliente.

11) Conclusiones y comentario del proyecto

El proyecto integró tres piezas: un MCP local de vuelos, un MCP remoto para hora (expuesto por Cloudflare Tunnel) y un MCP externo de un compañero para análisis de logs. Se probaron por separado y en conjunto (por ejemplo, “hora y estado”), confirmando que el chatbot puede orquestar varios servicios.

Se añadieron controles para que el sistema falle mejor: timeouts, manejo de errores 502 de servicios externos, un healthcheck y parseo flexible de respuestas (date/time y timestamps con Z). Con esto, cuando algo upstream falla, el chatbot no se cae y muestra un error entendible.

Sobre red y túnel, los logs mostraron que cloudflared usa QUIC y que el flujo cliente → edge → túnel → FastAPI funciona. También se entendió por qué aparecen conexiones en TIME_WAIT y cómo liberar puertos ocupados (taskkill, cambiar de puerto, variables en .env). En loopback no hay tráfico físico de capa de enlace; el intercambio es TCP local.

En aplicación, quedaron claras tres cosas: cuándo usar JSON vs multipart (archivo) para el analizador de logs, la necesidad de enviar UTF-8 (evitando los archivos UTF-16 que genera PowerShell con >), y que conviene limitar CORS y/o proteger /get_time con un token si se publica.

Limitaciones y siguientes pasos: el quick tunnel de Cloudflare es temporal y no es para producción; depender de APIs externas (AviationStack) puede dar 429/5xx. A futuro conviene usar Named Tunnels con DNS propio, añadir autenticación (Bearer/JWT), reintentos con backoff y circuit-breaker, métricas (p. ej. Prometheus) y pruebas automáticas con validación de esquemas (Pydantic). En resumen, se cumplió la meta del curso: orquestar MCPs en distintas topologías y protocolos, y aprender a diagnosticar y endurecer servicios con herramientas y prácticas básicas.